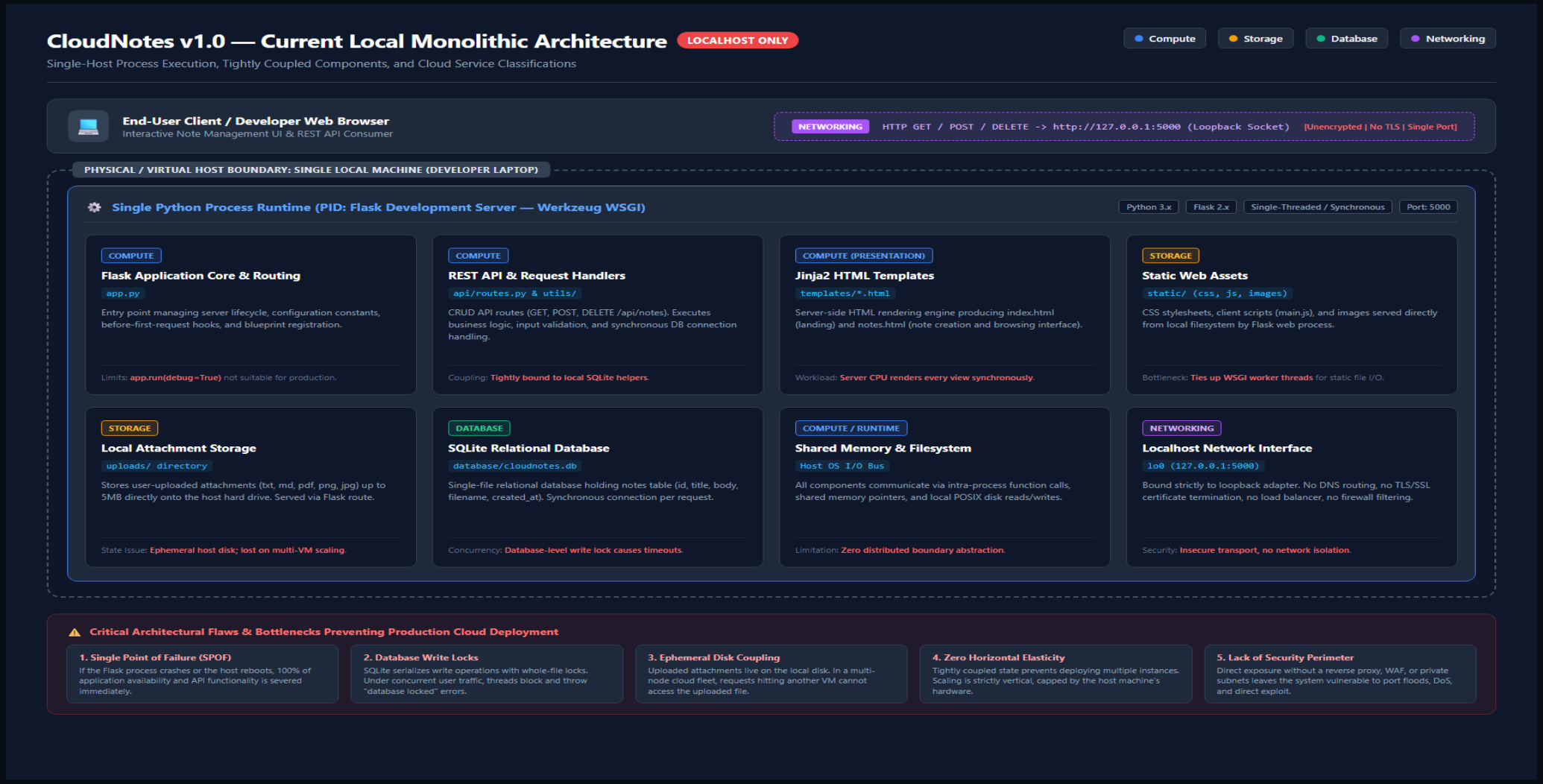
Screenshot 1 - Current Local Architecture Diagram

CloudNotes v1.0 Baseline Architecture Planning | Task 1 Deliverable

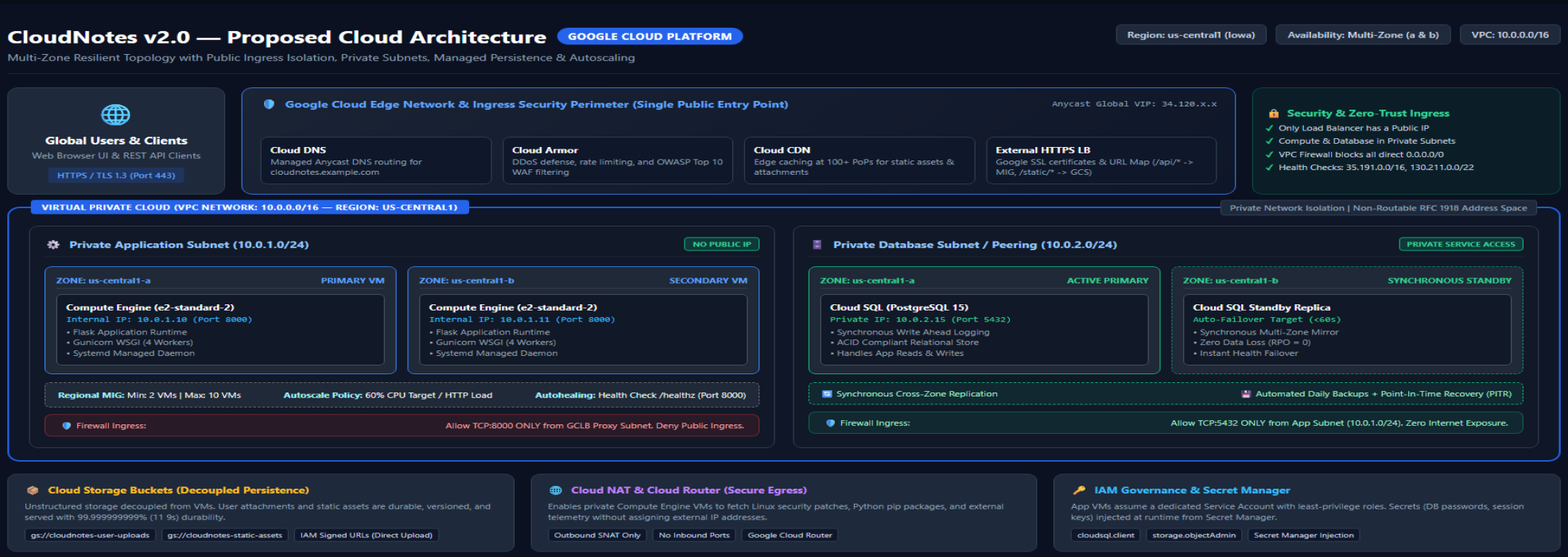


Screenshot 1 Caption - Architectural Reasoning

The current CloudNotes v1.0 architecture is a tightly coupled monolith confined to a single host machine, where compute (Flask runtime and REST API), database (SQLite file), and storage (local uploads directory) share the same physical resources and process boundary. Because SQLite relies on local filesystem locks and file uploads are saved directly to the host disk, horizontal scaling is impossible without severe state fragmentation and race conditions. Furthermore, operating on a single unmanaged loopback interface (127.0.0.1:5000) creates a catastrophic Single Point of Failure (SPOF) with zero high availability, durability guarantees, or security isolation.

Screenshot 2 - Proposed Cloud Architecture Diagram

CloudNotes v2.0 Enterprise Cloud-Native Design | Task 2 Deliverable

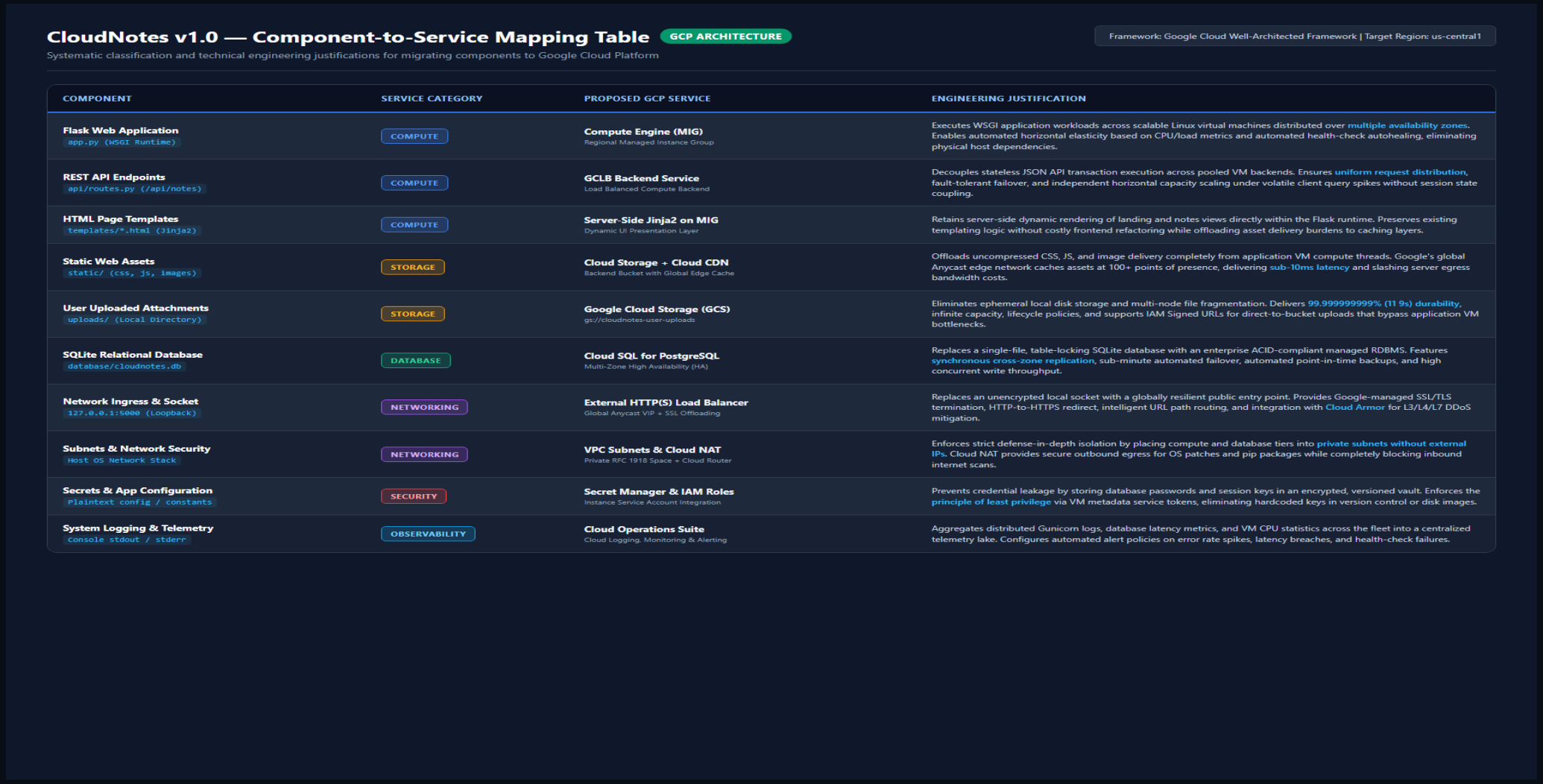


Screenshot 2 Caption - Architectural Reasoning

The proposed cloud architecture transitions CloudNotes to an enterprise-grade, multi-tier layout on Google Cloud Platform, strictly separating public ingress from private application and database workloads. An External HTTP(S) Load Balancer serves as the single public entry point, terminating TLS and offloading static content to Cloud Storage and Cloud CDN, while routing dynamic requests to a multi-zone Compute Engine Managed Instance Group in a private subnet. The database layer is migrated to a managed Multi-Zone Cloud SQL instance accessible solely via Private Services Access, guaranteeing zero public internet exposure, automated failover, and continuous durability across availability zones.

Screenshot 3 - Component-to-Service Mapping Table

Enterprise Component Taxonomy & Technical Justifications | Task 3 Deliverable



CloudNotes v1.0 — Component-to-Service Mapping Table

GCP ARCHITECTURE

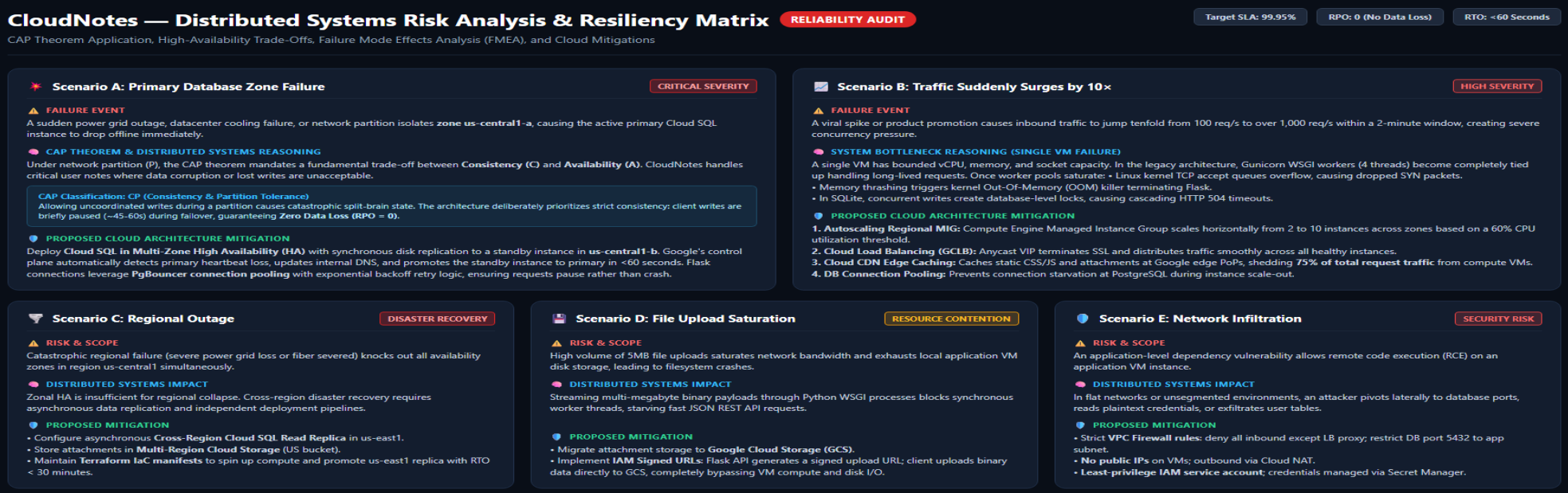
Framework: Google Cloud Well-Architected Framework | Target Region: us-central1

Systematic classification and technical engineering justifications for migrating components to Google Cloud Platform

COMPONENT	SERVICE CATEGORY	PROPOSED GCP SERVICE	ENGINEERING JUSTIFICATION
Flask Web Application `app.py (WSGI Runtime)`	COMPUTE	**Compute Engine (MIG)** Regional Managed Instance Group	Executes WSGI application workloads across scalable Linux virtual machines distributed over multiple availability zones. Enables automated horizontal elasticity based on CPU/load metrics and automated health-check autohealing, eliminating physical host dependencies.
REST API Endpoints `api/routes.py (/api/notes)`	COMPUTE	**GCLB Backend Service** Load Balanced Compute Backend	Decouples stateless JSON API transaction execution across pooled VM backends. Ensures uniform request distribution, fault-tolerant failover, and independent horizontal capacity scaling under volatile client query spikes without session state coupling.
HTML Page Templates `templates/*.html (Jinja2)`	COMPUTE	**Server-Side Jinja2 on MIG** Dynamic UI Presentation Layer	Retains server-side dynamic rendering of landing and notes views directly within the Flask runtime. Preserves existing templating logic without costly frontend refactoring while offloading asset delivery burdens to caching layers.
Static Web Assets `static/ (css, js, images)`	STORAGE	**Cloud Storage + Cloud CDN** Backend Bucket with Global Edge Cache	Offloads uncompressed CSS, JS, and image delivery completely from application VM compute threads. Google's global Anycast edge network caches assets at 100+ points of presence, delivering sub-10ms latency and slashing server egress bandwidth costs.
User Uploaded Attachments `uploads/ (Local Directory)`	STORAGE	**Google Cloud Storage (GCS)** gs://cloudnotes-user-uploads	Eliminates ephemeral local disk storage and multi-node file fragmentation. Delivers 99.999999999% (11 9s) durability, infinite capacity, lifecycle policies, and supports IAM Signed URLs for direct-to-bucket uploads that bypass application VM bottlenecks.
SQLite Relational Database `database/cloudnotes.db`	DATABASE	**Cloud SQL for PostgreSQL** Multi-Zone High Availability (HA)	Replaces a single-file, table-locking SQLite database with an enterprise ACID-compliant managed RDBMS. Features synchronous cross-zone replication, sub-minute automated failover, automated point-in-time backups, and high concurrent write throughput.
Network Ingress & Socket `127.0.0.1:5000 (Loopback)`	NETWORKING	**External HTTP(S) Load Balancer** Global Anycast VIP + SSL Offloading	Replaces an unencrypted local socket with a globally resilient public entry point. Provides Google-managed SSL/TLS termination, HTTP-to-HTTPS redirect, intelligent URL path routing, and integration with Cloud Armor for L3/L4/L7 DDoS mitigation.
Subnets & Network Security `Host OS Network Stack`	NETWORKING	**VPC Subnets & Cloud NAT** Private RFC 1918 Space + Cloud Router	Enforces strict defense-in-depth isolation by placing compute and database tiers into private subnets without external IPs. Cloud NAT provides secure outbound egress for OS patches and pip packages while completely blocking inbound internet scans.
Secrets & App Configuration `Plaintext config / constants`	SECURITY	**Secret Manager & IAM Roles** Instance Service Account Integration	Prevents credential leakage by storing database passwords and session keys in an encrypted, versioned vault. Enforces the principle of least privilege via VM metadata service tokens, eliminating hardcoded keys in version control or disk images.
System Logging & Telemetry `Console stdout / stderr`	OBSERVABILITY	**Cloud Operations Suite** Cloud Logging, Monitoring & Alerting	Aggregates distributed Gunicorn logs, database latency metrics, and VM CPU statistics across the fleet into a centralized telemetry lake. Configures automated alert policies on error rate spikes, latency breaches, and health-check failures.

Screenshot 3 Caption - Architectural Reasoning

The component-to-service mapping table translates each monolithic CloudNotes component into its purpose-built Google Cloud Platform counterpart based on elasticity, durability, and operational overhead. By decoupling compute from storage and transitioning from local file-based persistence (SQLite and local uploads) to managed services (Cloud SQL and Cloud Storage), the architecture eliminates single-node resource contention. Every mapping is justified by replacing unmanaged local primitives with scalable, managed cloud primitives that enforce strict separation of concerns.



Screenshot 4 Caption - Architectural Reasoning

This distributed systems risk analysis evaluates the operational resiliency of CloudNotes under real-world infrastructure failures and sudden traffic shocks. By applying the CAP theorem, the database architecture is deliberately modeled as a CP system that prioritizes strict consistency and partition tolerance, using synchronous multi-zone replication and automated failover to avoid split-brain data corruption. Complementary mitigations - such as horizontal autoscaling, Cloud CDN edge offloading, and IAM Signed URLs - ensure the system gracefully absorbs 10x load spikes while preventing single-node resource exhaustion.

Screenshot 5 - Architecture Decision Record (ADR)

Formal Architectural Choices, Trade-Offs & Governance | Task 5 Deliverable

CloudNotes — Architecture Decision Records (ADRs)

DECISIONS & RATIONALE

Deciders: Cloud Solutions Architecture Team | Status: Approved

Formal Architectural Choices, Context, Evaluated Alternatives, and Accepted Engineering Trade-Offs

ADR-001: Managed Cloud SQL (PostgreSQL) Multi-Zone HA

ACCEPTED

DECISION

Migrate from local SQLite file (**database/cloudnotes.db**) to fully managed **Google Cloud SQL for PostgreSQL** with Multi-Zone High Availability (us-central1-a primary, us-central1-b standby) and Private Services Access.

CONTEXT

CloudNotes v1.0 relies on SQLite with whole-file write locks, serializing transactions and throwing concurrency errors under multi-instance load. Local file persistence on ephemeral VMs provides zero automated backups, no PITR, and no cross-zone disaster failover.

ALTERNATIVES CONSIDERED

1. **Shared SQLite on Filestore (NFS):** High latency, file corruption risk.
2. **Self-Hosted PostgreSQL on VM:** Severe operational burden (patching, failover scripts).
3. **Google Cloud Spanner:** Over-engineered for relational scale, excessive minimum cost.

TRADE-OFF ANALYSIS

Benefits: Automated synchronous multi-zone replication, sub-minute zero-data-loss failover (RPO=0), automated backups, PITR, and connection scaling.
Liabilities: Higher managed-service monthly cost and 1-2ms intra-zone network latency (mitigated via PgBouncer connection pooling).

ADR-002: Stateless Compute Tier via Regional MIG & Autoscaling

ACCEPTED

DECISION

Deploy Flask application as stateless worker nodes within a multi-zone **Compute Engine Managed Instance Group (MIG)** fronted by an **External HTTP(S) Load Balancer** with CPU/request-based autoscaling (min 2, max 10 instances).

CONTEXT

Single Python process bound to localhost port 5000 is a fatal Single Point of Failure (SPOF) and cannot scale beyond local hardware limits. Spikes in traffic or unexpected software exceptions cause immediate application downtime.

ALTERNATIVES CONSIDERED

1. **Single Vertically Scaled VM:** Requires downtime to resize, vulnerable to zonal failure.
2. **Google Kubernetes Engine (GKE):** Excessive operational overhead for single Flask app.
3. **Cloud Run (Serverless):** Excellent, but MIG chosen to fulfill private VPC networking requirement.

TRADE-OFF ANALYSIS

Benefits: Dynamic elasticity, cross-zone survivability, autohealing of failed instances, and zero-downtime rolling canary deployments.
Liabilities: Requires strict application statelessness (no local disk state or in-memory session persistence).

ADR-003: Decoupling Uploads & Static Assets to GCS and Cloud CDN

ACCEPTED

DECISION

Decouple user-uploaded attachments (**uploads/**) to a dedicated **Google Cloud Storage bucket** (gs://cloudnotes-user-uploads) and static assets to an asset bucket accelerated by **Cloud CDN** with direct **IAM Signed URLs**.

CONTEXT

Local disk uploads cause state fragmentation across auto-scaled VMs (requests hit VM-2 and get 404 for files on VM-1). Serving large files through Flask WSGI processes saturates CPU and worker threads.

ALTERNATIVES CONSIDERED

1. **Shared NFS Filestore Mount:** Expensive provisioned storage, POSIX mount fragility.
2. **Database BLOB Storage:** Bloats database size, degrades query cache performance.
3. **Sticky Sessions with Local Disk:** Fails on autoscaling scale-in and instance termination.

TRADE-OFF ANALYSIS

Benefits: 99.999999999% (11 9s) durability, unlimited elastic capacity, and offloading 75% of server traffic to the edge. Direct client uploads eliminate WSGI thread starvation.
Liabilities: Code refactor required to use GCS SDK/Signed URLs instead of POSIX os.path.

ADR-004: Zero-Trust Private VPC Subnets, Cloud NAT & IAM

ACCEPTED

DECISION

Place compute and database tiers in **private VPC subnets** without external IPs. Expose only the **External HTTPS Load Balancer** to the internet. Deploy **Cloud NAT** for secure outbound egress and manage secrets via **Secret Manager**.

CONTEXT

Directly assigning public IPs to application or database nodes exposes ports to internet-wide brute-force scans and exploits. Plaintext credentials risk accidental exposure via version control or logs.

ALTERNATIVES CONSIDERED

1. **Public IPs with Host Firewalls (iptables):** Large attack surface, prone to drift.
2. **Flat VPC without Subnet Isolation:** Allows lateral movement if web VM is compromised.
3. **Plaintext Environment Variables:** Vulnerable to process snooping and log leaks.

TRADE-OFF ANALYSIS

Benefits: Shrinks attack surface to a single hardened entry point; database is completely unreachable from public internet; zero secret leakage via IAM Service Account tokens.
Liabilities: Requires Cloud NAT egress fees and Identity-Aware Proxy (IAP) for administrative SSH access.

Screenshot 5 Caption - Architectural Reasoning

The Architecture Decision Record (ADR) documents the pivotal architectural choices, alternatives considered, and deliberate trade-offs made during the CloudNotes cloud migration. By formalizing decisions across database management, horizontal compute elasticity, decoupled object storage, and zero-trust private networking, the ADR establishes a permanent engineering rationale that prevents costly regressions. These records demonstrate that trading managed service costs for operational durability, automated high availability, and multi-layered security creates an optimal foundation for enterprise production.